

PRACTITIONER COURSE

AI Control Architecture

Governing, securing, assuring, and containing enterprise AI — end to end.

~2.5 hours

12 modules · practitioner level

Open source

CC BY 4.0 · vendor-agnostic

Hands-on

assess a real AI use case end to end

Instructor: Ankush Chowdhary · Neo Control

How this course works



Who it's for

Security architects, AI governance & risk, privacy, compliance, internal audit, and the engineers building AI. Anyone who has to answer “is this AI under control?” and prove it.



What you'll be able to do

Inventory and risk-tier any AI use case, select the right controls, grade how strongly each boundary holds, collect audit-ready evidence, and run an assessment end to end.



How each module runs

A short concept, the failure it prevents, the controls that close it, and the evidence you collect. Every module ends with key takeaways.

Pair the course with the open specification, templates, and the executive pack — all free at aicontrolarchitecture.org.

Twelve modules, one arc

1 Why AI control

2 See • Decide • Do

3 Proving a boundary holds

4 Inventory & Identity (07–08)

5 Data & Input boundaries (09–10)

6 Output & Action control (11–12)

7 Human accountability (13)

8 Assurance, monitoring, recovery (14–16)

9 Risk tiering

10 Assurance & evidence

11 Running an assessment

12 Adoption, maturity & governance

MODULE 1 / 12

Why AI control

In this module

- Why AI risk is first a control problem
- The one claim the whole architecture rests on
- What enterprise AI failure actually looks like

Adopted faster than control can absorb

- Copilots are enabled, AI appears inside SaaS, RAG is wired into corporate knowledge, and agents begin to call tools and act.
- Most of this happens before anyone can answer the basic control questions.
- A policy states intent. It cannot, by itself, bound what a system can see, decide, or do.

SIX CONTROL QUESTIONS

- What AI exists?
- What can it see?
- What can it decide?
- What can it do?
- Who is accountable?
- How is failure evidenced & contained?

Probabilistic system, deterministic authority

- Deterministic authority — when AI retrieves a document, shapes a decision, or calls a tool, the effect is real: the record is updated, the message is sent.
- Probabilistic system — the actor exercising that authority is non-deterministic; capability emerges from composition, especially in agents.
- The risk is the pairing. Neither is dangerous alone; existing controls were not designed for the combination.

As AI gets better, the case for control gets stronger — more capability means more consequence and more autonomy.

Not science fiction — missing controls

- A RAG system surfaces a document the user was never entitled to
- A copilot summarises confidential context into the wrong place
- An AI recommendation becomes a decision with no accountable owner
- An agent calls a tool it should never have been able to call
- Logs are insufficient to reconstruct what the AI saw or did
- No one knows how to disable the capability quickly

THE PATTERN

- Every one is a missing or unproven control.
- Not a malicious model.
- This is what the ten pillars close.

KEY TAKEAWAYS

Why AI control



AI risk in the enterprise is first a control problem — not ethics, model quality, or policy alone.



The danger is deterministic authority granted to a probabilistic actor.



Every realistic failure traces back to a missing or unproven control.



Control is what turns intent into an applied, testable, evidenced boundary.

See • Decide • Do

In this module

- The three surfaces through which AI touches the enterprise
- Why risk rises as oversight falls away
- How consequence × autonomy sets severity

See, Decide, Do

- See — what data, context, and systems the AI can access and retrieve. Uncontrolled: a confidentiality and boundary failure.
- Decide — what decisions the AI's output influences. Uncontrolled: output silently becomes a decision, with no accountability.
- Do — what tools, actions, and workflows the AI can act on. Uncontrolled: the AI becomes an unbounded actor.

USE IT TO SCOPE

- Every use case is some mix of See / Decide / Do.
- Name the mix first — it drives everything after.
- A read-only copilot ≠ an action-capable agent.

Consequence × autonomy

- Consequence — how much the use case touches (See → Decide → Do).
- Autonomy — how little human oversight stands between the AI and effect.
- Severity is a function of both. Controls in the architecture graduate along both axes.
- The uncomfortable implication: more capable models are trusted with more consequence and more autonomy — so the need for bounded, provable, reversible authority rises with capability.

A control architecture is not a hedge against AI being bad at its job — it is what makes it safe to let AI be good at it.

KEY TAKEAWAYS

See • Decide • Do



An AI touches the enterprise in exactly three ways: what it can See, Decide, and Do.



Risk rises See → Decide → Do, and as human oversight falls away.



Severity = consequence × autonomy; controls graduate along both.



The case for control grows as AI capability grows.

Proving a boundary holds

In this module

- Why a claimed control is not a real control
- The boundary-source ladder: Declared → Enforced
- The claim you can make when AI is under control

How you know a boundary is real

The architecture grades every control by the strength of the basis it rests on. This is what keeps it honest.

Declared	Asserted in the assessment; self-attested.	01
Evidenced	Backed by configuration, documentation, or a registry.	02
Verified	Confirmed by a live check against the running system.	03
Enforced	An active control point blocks the disallowed behaviour inline.	04

A decision made on a Declared boundary is visibly weaker than one made on a Verified or Enforced one — and the gap is auditable.

What “under control” means

- We know what it can see, decide, and do.
- We have graded how strongly each boundary is held.
- A named human owns every consequential outcome.
- We can observe and reconstruct what it did.
- We can stop it — and we can recover.

WHY IT MATTERS

- A bounded, evidenced, reversible claim.
- Made before you grant authority — not after failure.
- This is the outcome every pillar serves.

KEY TAKEAWAYS

Proving a boundary holds



A claim that a control exists is not the same as the control existing.



Grade every boundary: Declared → Evidenced → Verified → Enforced.



Never let “we have a policy” masquerade as “the control holds.”



“Under control” = bounded, evidenced, and reversible.

Inventory & Identity

In this module

- See the AI and own it (Pillar 07)
- Give AI a bounded identity, not a human's (Pillar 08)
- The evidence that proves each

AI Inventory & Classification

You cannot control what you cannot see.

CONTROLS

- Maintain an inventory of all AI — copilots, SaaS features, RAG, agents, model APIs
- Assign a business owner and a technical owner
- Classify by AI pattern and assign a risk tier
- Record lifecycle status and reassessment triggers

THE FAILURE IT PREVENTS

Shadow AI and silently-enabled vendor AI: unmanaged exposure you can't govern, and audits you can't pass.

EVIDENCE YOU COLLECT

- Inventory record
- Owner records
- Risk-tier assignment
- Pattern & vendor classification

Identity & Access Control

Give AI a bounded identity, not a human's.

CONTROLS

- Give the AI / agent a distinct identity — never a shared human credential
- Apply least privilege; scope delegated authority explicitly
- Review AI access through existing IAM / PAM processes
- Log access and denied attempts

THE FAILURE IT PREVENTS

Excessive access, where one prompt can reach — and exfiltrate — data far beyond the use case's need.

EVIDENCE YOU COLLECT

- Identity record
- Access approval
- Delegated-authority record
- Access review evidence

KEY TAKEAWAYS

Inventory & Identity



If it isn't inventoried and owned, it isn't governed — start every programme here.



Classification and tier drive every downstream control.



AI acts under its own bounded identity, at least privilege — never a human's.



Delegated authority is explicit, scoped, and logged.

Data & Input boundaries

In this module

- Constrain what AI can retrieve and expose (Pillar 09)
- Manage injection and untrusted input (Pillar 10)
- How to test these boundaries

Data Boundary Control

Constrain what AI can retrieve and expose.

CONTROLS

- Map data sources and record the highest classification touched
- Enforce retrieval boundaries — RAG must respect user entitlements
- Review retention and training / reuse, especially for vendor AI
- Prevent cross-boundary leakage into outputs

THE FAILURE IT PREVENTS

Unauthorised retrieval — users seeing documents they were never entitled to; a confidentiality breach.

EVIDENCE YOU COLLECT

- Data-source map
- Classification record
- Entitlement / boundary config
- Retention & reuse review

Prompt & Input Control

Treat retrieved and tool content as untrusted.

CONTROLS

- Separate instructions from data; don't let content become commands
- Validate and constrain inputs; apply guardrails
- Run prompt-injection tests against real retrieval and tools
- Handle sensitive data entered into prompts

THE FAILURE IT PREVENTS

Prompt injection through retrieved content — untrusted text hijacking the model into leaking or acting.

EVIDENCE YOU COLLECT

- Injection test results
- Guardrail configuration
- Input-validation rules
- Sensitive-input handling record

KEY TAKEAWAYS

Data & Input boundaries



Bound what AI can retrieve — and make retrieval respect entitlements.



Record the highest data classification each use case can reach.



Treat all retrieved and tool content as untrusted input.



Prove the boundary with injection testing, not assertion.

Output & Action control

In this module

- Stop output silently becoming a decision (Pillar 11)
- Bound what AI can do — the actor problem (Pillar 12)
- Approval gates, blast radius, and logs

Output & Decision Control

Stop output silently becoming a decision.

CONTROLS

- Classify output types and define output-validation rules
- Separate the AI recommendation from the final decision
- Make human review meaningful — context, sources, override path
- Require grounding / citations where output drives action

THE FAILURE IT PREVENTS

A recommendation becoming a decision with no accountable owner — and ceremonial “rubber-stamp” review.

EVIDENCE YOU COLLECT

- Output-validation criteria
- Human-review evidence
- Decision evidence
- Correction / override record

Tool & Action Control

Bound what AI can do — the actor problem.

CONTROLS

- Inventory tools; classify actions by risk; apply least privilege
- Require approval gates for high-risk actions
- Validate tool parameters; set blast-radius limits
- Log tool calls and denied action attempts

THE FAILURE IT PREVENTS

Unauthorised tool use and approval bypass — an agent taking actions it should never have reached.

EVIDENCE YOU COLLECT

- Tool inventory
- Action classification
- Approval-gate config
- Tool & denied-action logs

KEY TAKEAWAYS

Output & Action control



Never let AI output become a decision without validation and an owner.



Human review must be meaningful, not ceremonial.



For agents, bound the actor: least privilege, approval gates, blast-radius limits.



Log what was done — and what was denied.

Human accountability

In this module

- Why accountability cannot be delegated to the AI
- Naming an owner for every consequential outcome
- When “no owner” is itself a finding

Human Accountability Model

An AI cannot own an outcome. A named human must.

CONTROLS

- Assign a decision owner for every consequential outcome
- Name reviewer roles, risk-acceptance owners, and incident owners
- Keep AI as informing judgment — never the accountable party
- Where no owner can be named, record it as a finding

THE FAILURE IT PREVENTS

Consequential decisions with nobody accountable to a customer, an auditor, or a regulator.

EVIDENCE YOU COLLECT

- Decision-owner records
- Reviewer authority record
- Risk-acceptance record
- Incident-owner record

KEY TAKEAWAYS

Human accountability



AI can inform judgment; it cannot be accountable for it.



A named human owns every consequential outcome, approval, and exception.



If you cannot name the owner, that gap is itself a finding.



Accountability is a control — document it, don't assume it.

Assurance, monitoring, recovery

In this module

- Prove the controls actually hold (Pillar 14)
- Observe, log, and reconstruct (Pillar 15)
- Stop it, contain it, recover (Pillar 16)

AI Assurance & Testing

A control you haven't tested is a control you're guessing at.

CONTROLS

- Run assurance tests before production for higher-risk AI
- Add adversarial testing where required
- Re-test (regression) on material model, prompt, or tool change
- Track findings to remediation or explicit risk acceptance

THE FAILURE IT PREVENTS

Controls that look designed on paper but don't operate when it matters.

EVIDENCE YOU COLLECT

- Assurance test plan
- Test results
- Adversarial test results
- Findings & remediation log

Monitoring, Logging & Evidence

If you can't reconstruct it, you can't defend it.

CONTROLS

- Define an event taxonomy; log what AI saw, produced, and triggered
- Make incidents reconstructable end to end
- Assemble evidence packages for high-risk use cases
- Monitor for drift, anomalies, and control failure

THE FAILURE IT PREVENTS

An incident you cannot reconstruct — no defensible audit trail of what the AI did.

EVIDENCE YOU COLLECT

- Event logs
- Reconstruction test
- Evidence package
- Monitoring coverage record

Incident Containment & Recovery

Assume it will fail — design the way back before you grant authority.

CONTROLS

- Provide a kill-switch / revocation path; test it
- Define blast-radius limits and rollback or compensation
- Write an incident containment plan; tabletop it for high-risk AI
- Define restart criteria and ongoing monitoring

THE FAILURE IT PREVENTS

An agent that cannot be stopped, and a failure with no recovery path.

EVIDENCE YOU COLLECT

- Kill-switch test
- Rollback / compensation test
- Incident plan & tabletop result
- Restart criteria

KEY TAKEAWAYS

Assurance, monitoring, recovery



The last three pillars make boundedness provable, observable, and reversible.



Test controls before production — and re-test on material change.



Log for reconstruction; assume you'll need to prove what happened.



Every action-capable AI needs a tested kill-switch and a way back.

Risk tiering

In this module

- Why control depth must scale with risk
- The five tiers and their drivers
- Triaging a use case in five questions

Control depth scales with risk

T1	Fast-track	Low-risk productivity, public data only	Minimal
T2	Standard	Enterprise data or vendor AI	Data & vendor review
T3	Enhanced	Influences decisions & records	Decision accountability
T4	Action-capable	Calls tools, triggers workflows	Containment & approval gates
T5	High-impact	Autonomous, regulated, hard to reverse	Full assurance & evidence

Avoid two failures: over-controlling low-risk AI (bureaucracy and avoidance) and under-controlling high-risk AI (unmanaged exposure).

Five questions set the tier

- Does it use enterprise, customer, employee, regulated, or sensitive data?
- Does it influence decisions, records, or communications?
- Can it call tools, trigger workflows, or modify records?
- Is vendor AI involved?
- Is it customer-facing, regulated, privileged, or hard to reverse?

THE RULE

- All No → Tier 1, fast-track.
- Data or vendor → at least Tier 2.
- Influences decisions → at least Tier 3.
- Can act → at least Tier 4.
- High-impact / irreversible → Tier 5.

KEY TAKEAWAYS

Risk tiering

✓ Control depth scales with risk — one process for all AI is the wrong answer.

✓ Five questions triage any use case into Tiers 1-5.

✓ Tier drives the minimum controls, templates, and evidence required.

✓ Re-tier on material change: new data, new tools, more autonomy.

Assurance & evidence

In this module

- The ACA evidence object and why it's portable
- Grading boundary strength and building evidence packages
- Being audit-ready by design

Proof that travels between tools

- Every control produces evidence in one portable shape — so a connector from one vendor produces proof another tool, or an auditor, can consume without translation.
- Evidence carries its own boundary state and verification state — it knows how strongly it was established.
- Integrity (a hash) and freshness make it defensible and current.

AN EVIDENCE OBJECT CARRIES

- control it proves
- subject: system, tenant, data class
- evidence type & how collected
- boundary state (Declared→Enforced)
- verification state & confidence
- source & collector
- collected-at & freshness
- integrity hash

From controls to an evidence package

- Grade each control by boundary source — and push high-risk controls toward Verified or Enforced.
- Assemble a per-use-case evidence package: risk assessment, control assessment, test results, decision and incident evidence.
- Match evidence depth to tier — a minimum evidence set per Tier 1–5 makes “enough proof” objective.
- One assessment then serves many obligations through the framework crosswalks.

AUDITORS WANT

- What controls apply, by tier.
- Whether they operate — with proof.
- Who owns each outcome.
- That you can reconstruct and recover.

Assurance & evidence



Evidence is part of the control — not paperwork after the fact.



One portable evidence shape lets proof travel across tools and auditors.



Grade boundary strength; push high-risk controls to Verified / Enforced.



Assemble tier-appropriate evidence packages; reuse across obligations.

Running an assessment

In this module

- The end-to-end assessment flow
- A worked example: a customer-support agent
- Turning the outcome into action

The assessment flow

1 Describe

the use case, pattern, and business purpose

2 Classify & tier

See / Decide / Do → Tier 1-5

3 Select controls

the graduated set for that tier

4 Collect & test

evidence from systems; assurance tests

5 Assess & grade

boundary strength; findings & owners

6 Report & prove

evidence package; crosswalk to obligations

A customer-support agent

- Describe: an agent that reads tickets and the knowledge base, drafts replies, and can update ticket status.
- Classify: it can See (tickets, KB), Decide (draft responses), and Do (update records) → action-capable.
- Tier: calls tools and modifies records → Tier 4.
- Controls: agent identity, tool inventory, action classification, approval gate on status changes, data boundary on KB, injection tests, logs, kill-switch.

THE OUTCOME

- A tier, a control set, and findings.
- Each boundary graded by strength.
- A named owner per outcome.
- An evidence package that proves it.

KEY TAKEAWAYS

Running an assessment



The flow is repeatable: describe → classify → select → collect → assess → prove.



See / Decide / Do plus the five questions gives you the tier fast.



Controls follow the tier — you don't apply everything to everything.



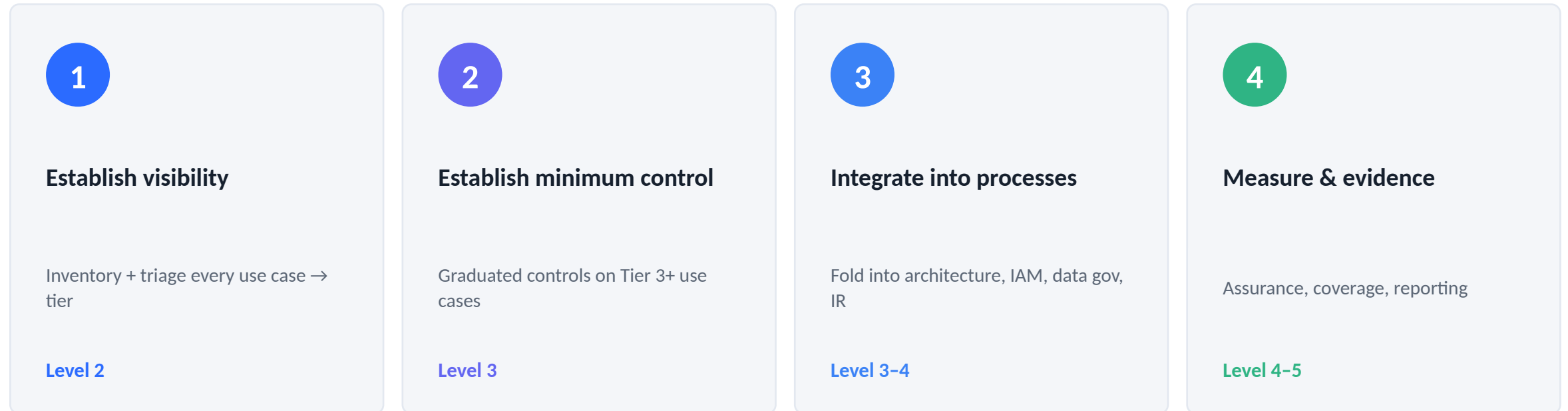
The deliverable is a graded, owned, evidenced assessment — reusable across obligations.

Adoption, maturity & governance

In this module

- A four-stage adoption path
- The five-level maturity model
- Operating the architecture over time

Four stages, not a big bang



Maturity: Ad Hoc → Visible → Integrated → Measurable → Adaptive. The goal at every level is the next one.

Governance that keeps it alive

- Make AI review part of normal operation — architecture, security, data governance, vendor risk, SDLC, incident response.
- Report coverage, findings, exceptions, and incidents to a governance forum.
- Time-bound every exception; an exception with no expiry becomes standing exposure.
- Reassess on material change — new data, model, tools, autonomy, or a near-miss.

METRICS TO REPORT

- Inventory completeness
- Control coverage by tier
- Findings & remediation
- Evidence & reconstruction readiness

KEY TAKEAWAYS

Adoption, maturity & governance



Adopt in four stages; each moves you one maturity level.



Integrate AI control into existing processes — don't bolt on a new one.



Govern with metrics, time-bound exceptions, and reassessment triggers.



Maturity is a direction of travel: start where you are.

COURSE COMPLETE

You can now put AI under control — and prove it.

- Inventory and risk-tier any AI use case
- Select the graduated controls for its tier
- Grade each boundary and collect audit-ready evidence
- Run an assessment end to end — and reuse it across obligations

• Put it to work with the open spec, templates, and quickstart — free at aicontrolarchitecture.org. •